

LLM에 RAG 추가하기



Contents

1 모델 다운로드

2 ask_llm.py 파일 만들고 코드 추가

3 코드 실행

4 pinky 라이브러리 관련 질문하면 pinky와 관련된 답을 하지 않는다

1 RAG 적용하기

1.1 pinky 라이브러리를 정리한 md 다운로드 후 워크스페이스에 위치

1.2 db 생성을 위한 임베딩 모델 다운로드

1.3 ChromaDB와langchain_text_splitters설치

1.4 ask_llm.py 파일을 ask_llm_rag.py로 복사하기

1.5 rag적용을 위한 모듈 import 및 모델 추가

1.6 ChromaDB 와 임베딩 모델 초기화 코드 추가

1.7 설정한 md 파일을 읽어서 ChromaDB로 DB 만들기

1.8 DB에서 질문과 유사한 chunk를 찾는 shearch 함수 생성

1.9 chat 함수에 search함수로 찾은 문서를 프롬프트로 만드는 부분 추가

1.10 전체 코드

1.11 동일한 질문을 해보면 약간 틀리지만 설명을 잘한다

1.12 보다 큰 4B 모델로 동작해보면 더 정확한 결과를 얻을 수 있다

LLM에 RAG 추가하기

Exported from Confluence · <https://pinkwink.atlassian.net>

모델 다운로드

Code

```
1 ollama pull hf.co/rippertnt/HyperCLOVAX-SEED-Text-Instruct-1.5B-Q4_K_M-GGUF:Q4_K_M
```

ask_llm.py 파일 만들고 코드 추가

- MODEL_NAME을 바꿔서 다른 모델로 설정할 수 있다

```
ask_llm.py × ▶ ▾ □ ⋮
ask_llm.py > ...
1 import requests
2
3 MODEL_NAME = "hf.co/rippertnt/HyperCLOVAX-SEED-Text-Instruct-1.5B-Q4_K_M-GGUF:Q4_K_M"
4
5 def ask_ollama(model, prompt):
6     resp = requests.post(
7         "http://localhost:11434/api/chat",
8         json={"model": model, "messages": [{"role": "user", "content": prompt}], "stream": False}
9     )
10    return resp.json()["message"]["content"]
11
12 def chat():
13     print(f"Model: {MODEL_NAME}")
14     print("exit 입력시 종료")
15
16     while True:
17         user_input = input("\n질문: ").strip()
18
19         if user_input.lower() in ["exit", "quit"]:
20             break
21
22         if not user_input: continue
23
24         answer = ask_ollama(MODEL_NAME, user_input)
25         print(f"답변: {answer}")
26
27 if __name__ == "__main__":
28     chat()
```

Code

```
1 import requests
2
3 MODEL_NAME = "hf.co/ripperint/HyperCLOVAX-SEED-Text-Instruct-1.5B-Q4_K_M-GGUF:Q4_K_
M"
4
5 def ask_ollama(model, prompt):
6     resp = requests.post(
7         "http://localhost:11434/api/chat",
8         json={"model": model, "messages": [{"role": "user", "content": prompt}], "st
ream": False}
9     )
10    return resp.json()["message"]["content"]
11
12 def chat():
13     print(f"Model: {MODEL_NAME}")
14     print("exit 입력시 종료")
15
16     while True:
17         user_input = input("\n질문: ").strip()
18
19         if user_input.lower() in ["exit", "quit"]:
20             break
21
22         if not user_input: continue
23
24         answer = ask_ollama(MODEL_NAME, user_input)
25         print(f"답변: {answer}")
26
27 if __name__ == "__main__":
28     chat()
```

코드 실행

Code

```
1 python3 ask_llm.py
```

```
(llm) mkh@mkh:~/ollama_ws$ python3 ask_llm.py
Model: hf.co/rippertnt/HyperCLOVAX-SEED-Text-Instruct-1.5B-Q4_K_M-G
GUF:Q4_K_M
exit 입력시 종료
```

질문 :

pinky 라이브러리 관련 질문하면 pinky와 관련된 답을 하지 않는다

```
(llm) mkh@mkh:~/ollama_ws$ python3 not_rag.py
Model: hf.co/rippertnt/HyperCLOVAX-SEED-Text-Instruct-1.5B-Q4_K_M-G
GUF:Q4_K_M
exit 입력시 종료
```

질문: Pinky 로봇의 모터를 제어하려면 어떤 라이브러리를 import 해야 하나요?

답변: pinky 로봇을 제어하기 위해서는 RPiUtils 라이브러리가 필요합니다. 이를 import 하여 로봇의 모든 기능을 사용할 수 있습니다.

```
```python
import RPi.GPIO as GPIO

GPIO 초기화
GPIO.setmode(GPIO.BCM)

모터 핀 연결 설정
pin = [23, 24]

모터 제어 (온 / 오프)
for p in pin:
 GPIO.setup(p, GPIO.OUT) # 출력으로 설정
 GPIO.output(p, True) # 켜짐

while True: # 무한 루프
 pass
...`
```

위 코드는 RPiUtils 라이브러리를 import 하여 GPIO를 초기화하고, 공격의 핀 연결을 정의합니다. 이후에는 로봇의 모든 기능을 제어할 수 있습니다.

단, 이 코드는 비전문적인 예시이며 실제로 로봇의 모터를 제어하기 위해서는 해당 로봇의 하드웨어 및 소프트웨어에 대한 이해가 필요합니다.

## RAG 적용하기

pinky 라이브러리를 정리한 md 다운로드 후 워크스페이스에 위치

db 생성을 위한 임베딩 모델 다운로드

## Code

```
1 ollama pull mxbai-embed-large
```

## ChromaDB와 langchain\_text\_splitters 설치

- **ChromaDB**: 텍스트 임베딩을 저장·검색하는 벡터 데이터베이스
- **langchain\_text\_splitters**: 긴 텍스트를 LLM/RAG에 쓰기 좋게 여러 청크로 분할하는 유틸리티

## Code

```
1 pip install chromadb
2 pip install langchain-text-splitters
```

## ask\_llm.py 파일을 ask\_llm\_rag.py로 복사하기

ask\_llm\_rag.py X ask\_llm.py

▶ ▢ ...

ask\_llm\_rag.py > chat

```
1 import requests
2
3 MODEL_NAME = "hf.co/rippertnt/HyperCLOVAX-SEED-Text-Instruct-1.5B-Q4_K_M-GGUF:Q4_K_M"
4
5 def ask_ollama(model, prompt):
6 resp = requests.post(
7 "http://localhost:11434/api/chat",
8 json={"model": model, "messages": [{"role": "user", "content": prompt}], "stream": False}
9)
10 return resp.json()["message"]["content"]
11
12 def chat():
13 print(f"Model: {MODEL_NAME}")
14 print("exit 입력시 종료")
15
16 while True:
17 user_input = input("\n질문: ").strip()
18
19 if user_input.lower() in ["exit", "quit"]:
20 break
21
22 if not user_input: continue
23
24 answer = ask_ollama(MODEL_NAME, user_input)
25 print(f"답변: {answer}")
26
27 if __name__ == "__main__":
28 chat()
```

## rag적용을 위한 모듈 import 및 모델 추가

```
ask_llm_rag.py > ...
1 import os
2 import requests
3 import chromadb
4 from chromadb.utils import embedding_functions
5 from langchain_text_splitters import RecursiveCharacterTextSplitter
6
7 # =====
8 # [설정] 모델 및 파일 경로 설정
9 # =====
10 MODEL_NAME = "hf.co/rippertnt/HyperCLOVAX-SEED-Text-Instruct-1.5B-Q4_K_M-GGUF:Q4_K_M"
11 EMBED_MODEL = "mxbai-embed-large"
12 MD_FILE_PATH = "pinky_library.md"
13 DB_PATH = "./pinky_rag_db"
14
```

## ChromaDB 와 임베딩 모델 초기화 코드 추가

```
15 # =====
16 # 1) ChromaDB 및 임베딩 초기화
17 # =====
18 # 로컬 디스크에 데이터를 영구 저장하는 ChromaDB 클라이언트 생성
19 chroma = chromadb.PersistentClient(path=DB_PATH)
20
21 # Ollama의 임베딩 API를 사용하기 위한 설정
22 # (텍스트 -> 숫자 벡터 변환 담당)
23 embed_fn = embedding_functions.OllamaEmbeddingFunction(
24 model_name=EMBED_MODEL,
25 url="http://localhost:11434/api/embeddings"
26)
27
28 # 컬렉션 가져오기 또는 생성 (기존 DB 유지)
29 collection = chroma.get_or_create_collection(
30 name="pinky_docs",
31 embedding_function=embed_fn
32)
33 |
```

## 설정한 md 파일을 읽어서 ChromaDB로 DB 만들기



```

37 # 컬렉션에 데이터가 없으면 문서를 로딩하고 저장합니다.
38 if collection.count() == 0:
39 if os.path.exists(MD_FILE_PATH):
40 print("문서 로딩 및 청킹 중...")
41
42 # 마크다운 파일 읽기
43 with open(MD_FILE_PATH, "r", encoding="utf-8") as f:
44 docs = f.read()
45
46 # 문서를 작은 조각(Chunk)으로 자르기 위한 객체 생성
47 # chunk_size: 한 조각당 약 500자
48 # chunk_overlap: 문맥이 끊기지 않도록 앞뒤 내용을 50자씩 겹치게 함
49 splitter = RecursiveCharacterTextSplitter(
50 chunk_size=500,
51 chunk_overlap=50
52)
53
54 # 텍스트 분할 수행
55 chunks = splitter.split_text(docs)
56
57 # =====
58 # 3) 벡터 DB에 데이터 삽입
59 # =====
60 print(f"{len(chunks)}개 청크 DB 저장 중...")
61 for i, chunk in enumerate(chunks):
62 # ChromaDB에 데이터 추가 (텍스트는 자동으로 임베딩됨)
63 collection.add(
64 ids=[f"id_{i}"], # 각 문서 조각의 고유 ID
65 documents=[chunk] # 실제 텍스트 내용
66)
67 print("문서 저장 완료!")
68 else:
69 print("경고: MD 파일이 없습니다.")
70 else:
71 print("기존 DB를 로드했습니다.")

```

## DB에서 질문과 유사한 chunk를 찾는 shearch 함수 생성

- **top\_k**: 질문과 의미적으로 가장 가까운 상위 3개의 문서 조각(Chunk)을 찾아냄
  - 만약 **top\_k=1** 로 설정하면 가장 비슷한 것 딱 1개만 가져오고, **top\_k=5** 로 하면 5개를 가져와서 더 많은 정보를 참고

```

73 # =====
74 # 4) 검색 및 API 호출 함수 정의
75 # =====
76 def search(query, top_k=3):
77 """
78 사용자 질문(Query)과 가장 유사한 문서 조각을 검색합니다.
79 top_k: 검색할 문서 개수 (기본 3개)
80 """
81 result = collection.query(
82 query_texts=[query],
83 n_results=top_k
84)
85 # 검색 결과가 있으면 문서 리스트 반환
86 if result["documents"]:
87 return result["documents"][0]
88 return []
89
90 def ask_ollama(model, prompt):
91 resp = requests.post(
92 "http://localhost:11434/api/chat",
93 json={"model": model, "messages": [{"role": "user", "content": prompt}], "stream": False}
94)
95 return resp.json()[0]["message"]["content"]
96
97

```

## chat 함수에 search함수로 찾은 문서를 프롬프트로 만드는 부분 추가

```

100 def chat():
101 user_input = input("질문: ")
102
103 # 종료 조건 처리
104 if user_input.lower() in ["exit", "quit"]:
105 print("종료합니다.")
106 break
107
108 if not user_input: continue
109
110 # 1. 문서 검색 (Retrieval)
111 retrieved_docs = search(user_input, top_k=3)
112
113 # 검색된 문서 조각들을 하나의 문자열로 합침
114 context = "\n\n".join(retrieved_docs)
115
116 # 2. 프롬프트 구성 (Augmentation)
117 # 검색된 컨텍스트(Context)를 질문과 함께 모델에게 전달
118 final_prompt = f"""
119 다음 문서를 참고해서 질문에 답해줘.
120
121 [Context]
122 {context}
123
124 [Question]
125 {user_input}
126 """
127
128 # 3. 답변 생성 (Generation)
129 print("...생성 중")
130 answer = ask_ollama(MODEL_NAME, final_prompt)
131 print(f"답변: {answer}")
132
133 if __name__ == "__main__":
134 chat()

```

## 전체 코드

## Code

```

1 import os
2 import requests
3 import chromadb
4 from chromadb.utils import embedding_functions
5 from langchain_text_splitters import RecursiveCharacterTextSplitter
6
7 # =====
8 # [설정] 모델 및 파일 경로 설정
9 # =====
10 MODEL_NAME = "hf.co/rippertnt/HyperCLOVAX-SEED-Text-Instruct-1.5B-Q4_K_M-GGUF:Q4_K_M"
11
12 EMBED_MODEL = "mxbai-embed-large"
13 MD_FILE_PATH = "pinky_library.md"
14 DB_PATH = "./pinky_rag_db"
15
16 # =====
17 # 1) ChromaDB 및 임베딩 초기화
18 # =====
19 # 로컬 디스크에 데이터를 영구 저장하는 ChromaDB 클라이언트 생성
20 chroma = chromadb.PersistentClient(path=DB_PATH)
21
22 # Ollama의 임베딩 API를 사용하기 위한 설정
23 # (텍스트 -> 숫자 벡터 변환 담당)
24 embed_fn = embedding_functions.OllamaEmbeddingFunction(
25 model_name=EMBED_MODEL,
26 url="http://localhost:11434/api/embeddings"
27)
28
29 # 컬렉션 가져오기 또는 생성 (기존 DB 유지)
30 collection = chroma.get_or_create_collection(
31 name="pinky_docs",
32 embedding_function=embed_fn
33)
34
35 # =====
36 # 2) 문서 로딩 및 청킹 (DB가 비어있을 때만)
37 # =====
38 # 컬렉션에 데이터가 없으면 문서를 로딩하고 저장합니다.
39 if collection.count() == 0:
40 if os.path.exists(MD_FILE_PATH):
41 print("문서 로딩 및 청킹 중...")
42
43 # 마크다운 파일 읽기
44 with open(MD_FILE_PATH, "r", encoding="utf-8") as f:
45 docs = f.read()
46
47 # 문서를 작은 조각(Chunk)으로 자르기 위한 객체 생성
48 # chunk_size: 한 조각당 약 500자
49 # chunk_overlap: 문맥이 끊기지 않도록 앞뒤 내용을 50자씩 겹치게 함
50 splitter = RecursiveCharacterTextSplitter(
51 chunk_size=500,

```

```

51 chunk_overlap=50
52)
53
54 # 텍스트 분할 수행
55 chunks = splitter.split_text(docs)
56
57 # =====
58 # 3) 벡터 DB에 데이터 삽입
59 # =====
60 print(f"{len(chunks)}개 청크 DB 저장 중...")
61 for i, chunk in enumerate(chunks):
62 # ChromaDB에 데이터 추가 (텍스트는 자동으로 임베딩됨)
63 collection.add(
64 ids=[f"id_{i}"], # 각 문서 조각의 고유 ID
65 documents=[chunk] # 실제 텍스트 내용
66)
67 print("문서 저장 완료!")
68 else:
69 print("경고: MD 파일이 없습니다.")
70 else:
71 print("기존 DB를 로드했습니다.")
72
73 # =====
74 # 4) 검색 및 API 호출 함수 정의
75 # =====
76 def search(query, top_k=3):
77 """
78 사용자 질문(Query)과 가장 유사한 문서 조각을 검색합니다.
79 top_k: 검색할 문서 개수 (기본 3개)
80 """
81 result = collection.query(
82 query_texts=[query],
83 n_results=top_k
84)
85 # 검색 결과가 있으면 문서 리스트 반환
86 if result["documents"]:
87 return result["documents"][0]
88 return []
89
90 def ask_ollama(model, prompt):
91 resp = requests.post(
92 "http://localhost:11434/api/chat",
93 json={"model": model, "messages": [{"role": "user", "content": prompt}], "stream": False}
94)
95 return resp.json()["message"]["content"]
96
97 # =====
98 # 5) 메인 챗봇 루프 (Chat Loop)
99 # =====
100 def chat():
101 print(f"Model: {MODEL_NAME}")
102 print("종료하려면 'exit' 입력")
103

```

```

104 while True:
105 # 사용자 입력 받기
106 user_input = input("\n질문: ").strip()
107
108 # 종료 조건 처리
109 if user_input.lower() in ["exit", "quit"]:
110 print("종료합니다.")
111 break
112
113 if not user_input: continue
114
115 # 1. 문서 검색 (Retrieval)
116 retrieved_docs = search(user_input, top_k=3)
117
118 # 검색된 문서 조각들을 하나의 문자열로 합침
119 context = "\n\n".join(retrieved_docs)
120
121 # 2. 프롬프트 구성 (Augmentation)
122 # 검색된 컨텍스트(Context)를 질문과 함께 모델에게 전달
123 final_prompt = f"""
124 다음 문서를 참고해서 질문에 답해줘.
125
126 [Context]
127 {context}
128
129 [Question]
130 {user_input}
131 """
132
133 # 3. 답변 생성 (Generation)
134 print(" ...생성 중")
135 answer = ask_ollama(MODEL_NAME, final_prompt)
136 print(f"답변: {answer}")
137
138 if __name__ == "__main__":
139 chat()

```

**동일한 질문을 해보면 약간 틀리지만 설명을 잘한다**

질문: Pinky 로봇의 모터를 제어하려면 어떤 라이브러리를 import 해야 하나요?

...생성 중

답변: Pinky 로봇의 모터를 제어하려면 `pinkylib` 라이브러리를 import 해야 합니다. 이 라이브러리에는 모터 제어와 관련된 다양한 클래스 및 메서드가 포함되어 있습니다.

```
```python
```

```
from pinkylib import Motor, LCD # 필요한 모듈만 가져오는 것이 좋습니다.
```

```
```
```

이렇게 하면 Pinky 로봇의 모터를 제어하는 코드를 작성할 수 있습니다.

질문: LCD 화면을 제어하기 위해 사용하는 라이브러리 이름 알려줘.

...생성 중

답변: LCD 화면을 제어하기 위해 사용하는 라이브러리는 `pinky`입니다. 이 라이브러리를 통해 LCD 모듈을 관리하고 다양한 기능을 사용할 수 있습니다.

- `set\_backlight(value)`: 밝기를 설정합니다.
- `img\_show(img)`: 이미지를 LCD에 출력합니다.
- `clear()`: 화면을 초기화하여 검은 빈 화면으로 만듭니다.

**보다 큰 4B 모델로 동작해보면 더 정확한 결과를 얻을 수 있다**

Code

```
1 ollama pull qwen3:4b
```

질문 : Pinky에 대해서 설명해 줘

...생성 중

^[[A답변 : Pinky는 모터 및 카메라를 통한 로봇 제어를 지원하는 플랫폼입니다. 이 플랫폼을 통해 다음과 같은 기능을 구현할 수 있습니다:

- **\*\*모터 제어\*\***:

- ``enable_motor()``로 모터를 활성화하고, ``disable_motor()``로 비활성화합니다.

- ``move(L, R)``로 왼쪽 모터(L)와 오른쪽 모터(R)의 속도를 설정하여 이동합니다. (음수는 후진, 0은 정지)

- ``stop()``으로 주행 중 정지하고, ``close()``로 모터 사용을 종료합니다.

- **\*\*카메라 제어\*\***:

- ``start(width=640, height=480)``로 카메라를 활성화하고, 가로 640, 세로 480의 해상도로 설정할 수 있습니다.

예시 코드에서는 모터를 활성화한 후 1초 동안 이동한 후 정지하고 비활성화하는 과정을 보여줍니다. 이와 같이 Pinky는 로봇의 이동 제어와 시각 정보 수집을 통합적으로 관리할 수 있는 플랫폼으로 활용됩니다.

질문 : Pinky 로봇의 모터를 제어하려면 어떤 라이브러리를 import 해야 하나요?

...생성 중

답변 : Pinky 로봇의 모터를 제어하려면 ``pinkylib`` 라이브러리를 import 해야 합니다.

**\*\*정답\*\***: ``pinkylib``

**\*\*해설\*\***:

문맥에서 Motor 클래스는 ``from pinkylib import Motor``로 불러와 사용됩니다. 예시 코드에서도 ``pinkylib`` 모듈을 통해 모터 제어 기능을 제공하는 클래스를 사용하고 있습니다. 따라서 모터 제어를 위해 반드시 import해야 하는 라이브러리는 ``pinkylib``입니다.

질문 : LCD 화면을 제어하기 위해 사용하는 라이브러리 이름 알려줘.



## PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

[https://www.youtube.com/@pinklab\\_studio](https://www.youtube.com/@pinklab_studio)